

TELEGRAM FRAUD DETECTOR — COMPLETE HANDOVER



PROJECT OVERVIEW

The **Telegram Fraud Detector** is a Python-based system that detects phishing, scam, and malicious URLs/messages using:

- Rule-based detection (40+ signals)
 - Threat intelligence feeds
 - Machine learning model
 - Behavioral + structural analysis
-



PROJECT OBJECTIVE

👉 Detect:

- Phishing links
- Scam messages
- Malicious domains
- Social engineering attacks

👉 Platforms targeted:

- Telegram
 - WhatsApp
 - SMS / Email
-



PROJECT STRUCTURE

From your folder:

fraud-detector/

|

```
├── data/
│   ├── openphish.txt
│   ├── scam_patterns.txt
│   ├── threatfox.json
│   ├── url_dataset.csv
│   └── urlhaus.csv
├── detector/
│   ├── scam_detector.py
│   └── __pycache__/
├── main.py
├── ml_model.py
├── phishing_model.pkl
├── requirements.txt
├── tempCodeRunnerFile.py
└── Fraud Detector Handover PDFs
```

FILE-BY-FILE EXPLANATION

◆ 1. **main.py**

Role:

Entry point of the system

✓ Responsibilities:

- Takes user input
 - Calls `analyze_message()`
 - Displays:
 - Risk Score
 - Reasons
-

◆ 2. **detector/scam_detector.py** ★ (CORE ENGINE)

Role:

Main detection logic

✓ Contains:

- 40+ detection signals
 - Scoring system
 - API integrations
 - Feature engineering
 - ML prediction
-

◆ 3. `ml_model.py`

Role:

Model training script

✓ Responsibilities:

- Train phishing classifier
 - Use dataset (`url_dataset.csv`)
 - Save model → `phishing_model.pkl`
-

◆ 4. `phishing_model.pkl`

Role:

Pre-trained ML model

✓ Used for:

- Predict phishing based on:
 - URL length
 - dots
 - hyphens
 - digits
 - HTTPS presence

◆ 5. **data/** FOLDER (THREAT INTELLIGENCE)

openphish.txt

- Live phishing URLs
 - Used for direct matching
-

urlhaus.csv

- Malware URLs dataset
 - Extracts URLs for detection
-

threatfox.json

- IOC (Indicators of Compromise)
 - Domains + malicious links
-

scam_patterns.txt

- Known scam message templates
-

url_dataset.csv

- Training dataset for ML model
-

DETECTION ENGINE (INSIDE scam_detector.py)

LAYER 1: MESSAGE ANALYSIS

Signal	Purpose
Keywords	Detect urgency, reward
Scam patterns	Known fraud templates
Jaccard similarity	Similar scam text
URL encoding	Obfuscation detection

LAYER 2: URL EXTRACTION

- Extract URLs using regex
 - Detect hidden URLs
 - Detect multiple domains
-

LAYER 3: STRUCTURAL ANALYSIS

Signal	Purpose
URL length	Long phishing links
Numbers	Random domains
Dots	Subdomain tricks
Entropy	Random strings
Path depth	Fake navigation

LAYER 4: DOMAIN ANALYSIS

Signal	Purpose
Suspicious TLD	.xyz, .top
Domain popularity	unknown domains
Domain age	new domains
DNS failure	fake domains

LAYER 5: BEHAVIOR ANALYSIS

Signal	Purpose
Redirect detection	hidden links
Domain change	phishing redirection
Short URL	masked links

LAYER 6: BRAND ATTACK DETECTION

Signal	Purpose
Brand impersonation	amazon-logging
Typosquatting	amazn
Visual similarity	g00gle

LAYER 7: NETWORK ANALYSIS

Signal	Purpose
IP in URL	suspicious
AbuseIPDB	malicious IP
DNS resolution	invalid domains

LAYER 8: THREAT INTELLIGENCE

Source	Purpose
OpenPhish	phishing URLs
URLhaus	malware
PhishTank	verified phishing
ThreatFox	IOC database
Google Safe Browsing	real-time threat

👉 If matched → **score = 100 (immediate)**

LAYER 9: MACHINE LEARNING

Features used:

- URL length
- dot count
- hyphen count
- HTTPS presence
- IP presence



LAYER 10: INTELLIGENCE LOGIC

Feature	Purpose
Multi-signal boost	combine signals
Score grouping	avoid over-scoring
Weak score cap	reduce noise



SCORING SYSTEM

◆ Score Types

Type	Description
score	strong signals
weak_score	low confidence
structure_score	URL complexity
brand_score	brand attacks

◆ Score Limits

weak_score $\leq$ 40
structure_score $\leq$ 40
brand_score $\leq$ 50

◆ Final Score

Total Score = score + weak + structure + brand



FINAL CLASSIFICATION

Score	Result
≥ 80	HIGH RISK
50–79	MEDIUM RISK
< 50	LOW RISK



DEBUG FEATURES

Your system prints:

- Extracted URLs
- Domain
- Entropy
- Redirect count

👉 Helps debugging & testing



DEPENDENCIES (`requirements.txt`)

Likely includes:

- `requests`
 - `tldextract`
 - `dnspython`
 - `joblib`
 - `scikit-learn`
-



CURRENT LIMITATIONS



1. API Dependency

- WHOIS API (paid/limited)
 - Google Safe Browsing API
-



2. Performance

- Multiple API calls slow system
-



3. Score Overlap

- Multiple signals detect same thing
-



4. Offline Limitations

- APIs fail without internet
-



FUTURE IMPROVEMENTS



HIGH IMPACT

- Enable context-aware detection
- Cache API results
- Add URL screenshot detection
- Add blacklist auto-update

ADVANCED

- Deep learning model
 - Browser extension
 - Telegram bot integration
-

WORKFLOW SUMMARY

Input Message



Text Analysis



URL Extraction



Structure Analysis



Domain Analysis



Behavior Analysis



Threat Intelligence



ML Prediction



Score Aggregation



Final Verdict

FINAL CONCLUSION

 Your system:

- ✓ Uses 40+ signals
- ✓ Combines rule-based + ML
- ✓ Integrates real threat intelligence
- ✓ Handles phishing realistically



SHORT SUMMARY (FOR REPORT)

The Telegram Fraud Detector is a multi-layer phishing detection system that analyzes messages and URLs using rule-based techniques, machine learning, and threat intelligence feeds. It evaluates over 40 signals including URL structure, domain characteristics, behavioral patterns, and known threat databases to classify links into low, medium, or high risk categories.
